TRINITY C2

INSY7315 - PART 2

James Marshall (ST10434366), Lyle William Richards (ST10322521), Sky Martin (ST10286905), Connaire Myall (ST10297849), and Simon Slingerland (ST10438897)

05 October 2026

0x0001

OVERVIEW

Key features & functionality alignment

System Architecture & design decisions

GitHub version control

Automated testing and action pipeline

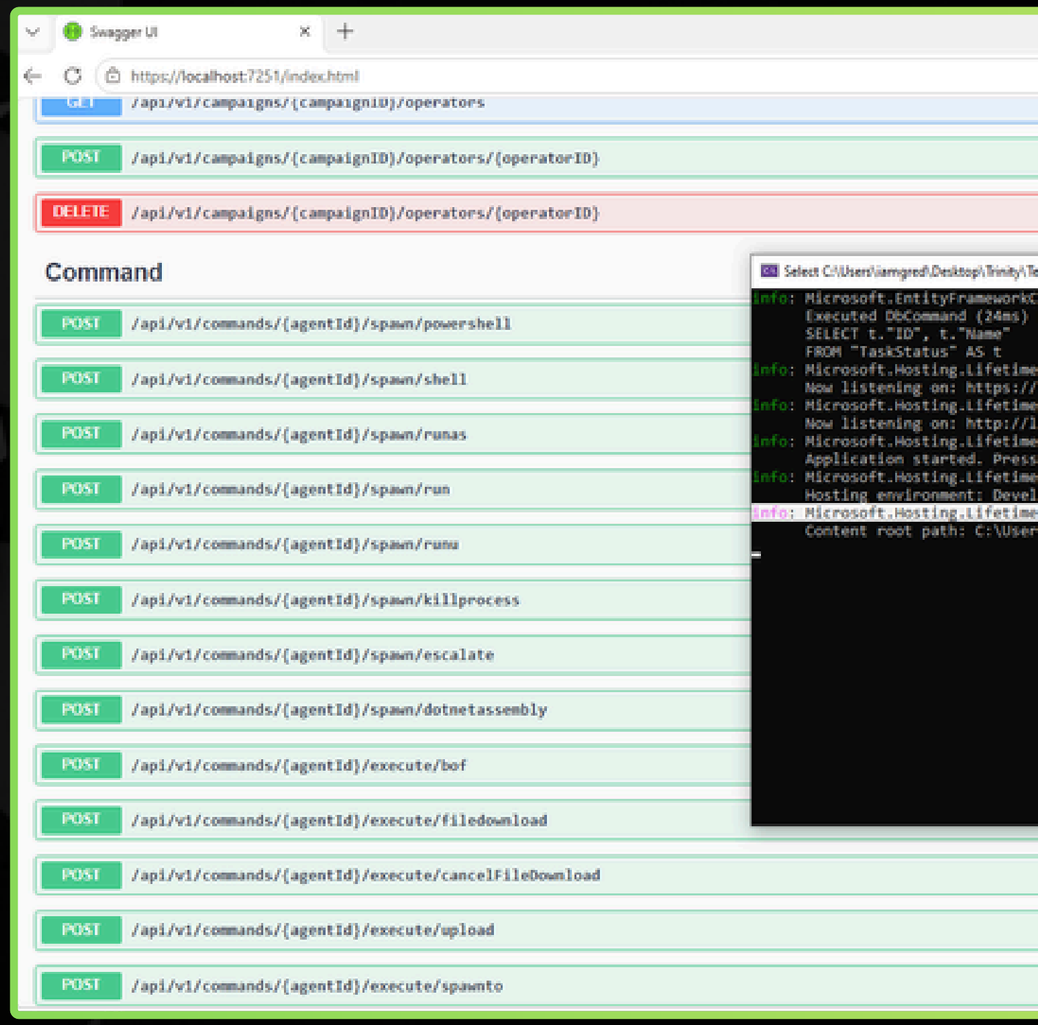
Demonstration

Code Walkthrough

KEY FEATURES & FUNCTIONALITY

Team server

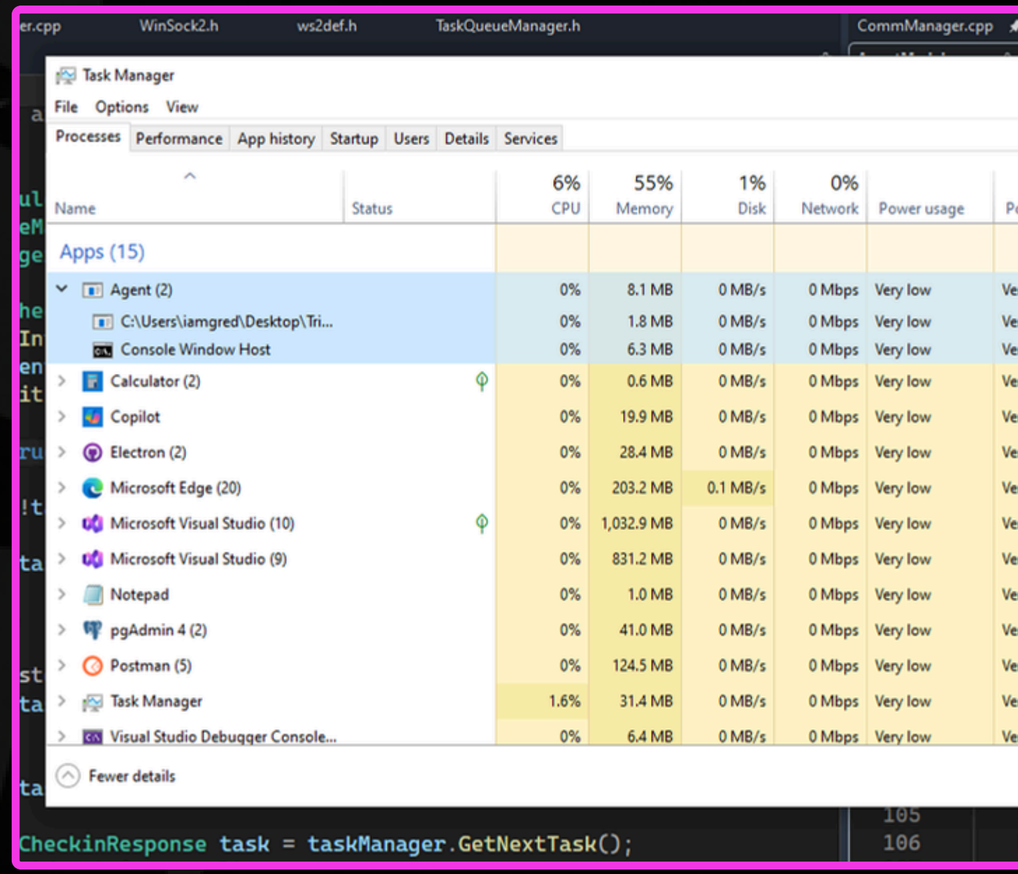
- **Listener Management:** Hosting and configuring modular endpoints (Listeners) configured to handle and decode incoming traffic across different protocols (HTTP, and HTTPS).
- **Payload Generation:** Compiling or packaging agent binaries dynamically based on operator defined specifications such as target architecture and hosts.
- **Session Registration:** Identifies brand new agents registers host metadata, and spins up active sessions.
- **Checkin Processing:** Decodes and deserializes incoming requests, handles agent registration, and returns pending tasks for execution.
- **Task Management:** Maintains an internal queue on the server side to store operator commands until the agent checks in to receive them.
- **Real-Time Output Dispatcher:** Uses a SignalR Hub to instantly stream command results from the Teamserver queue directly back to the Blazor client as they arrive.



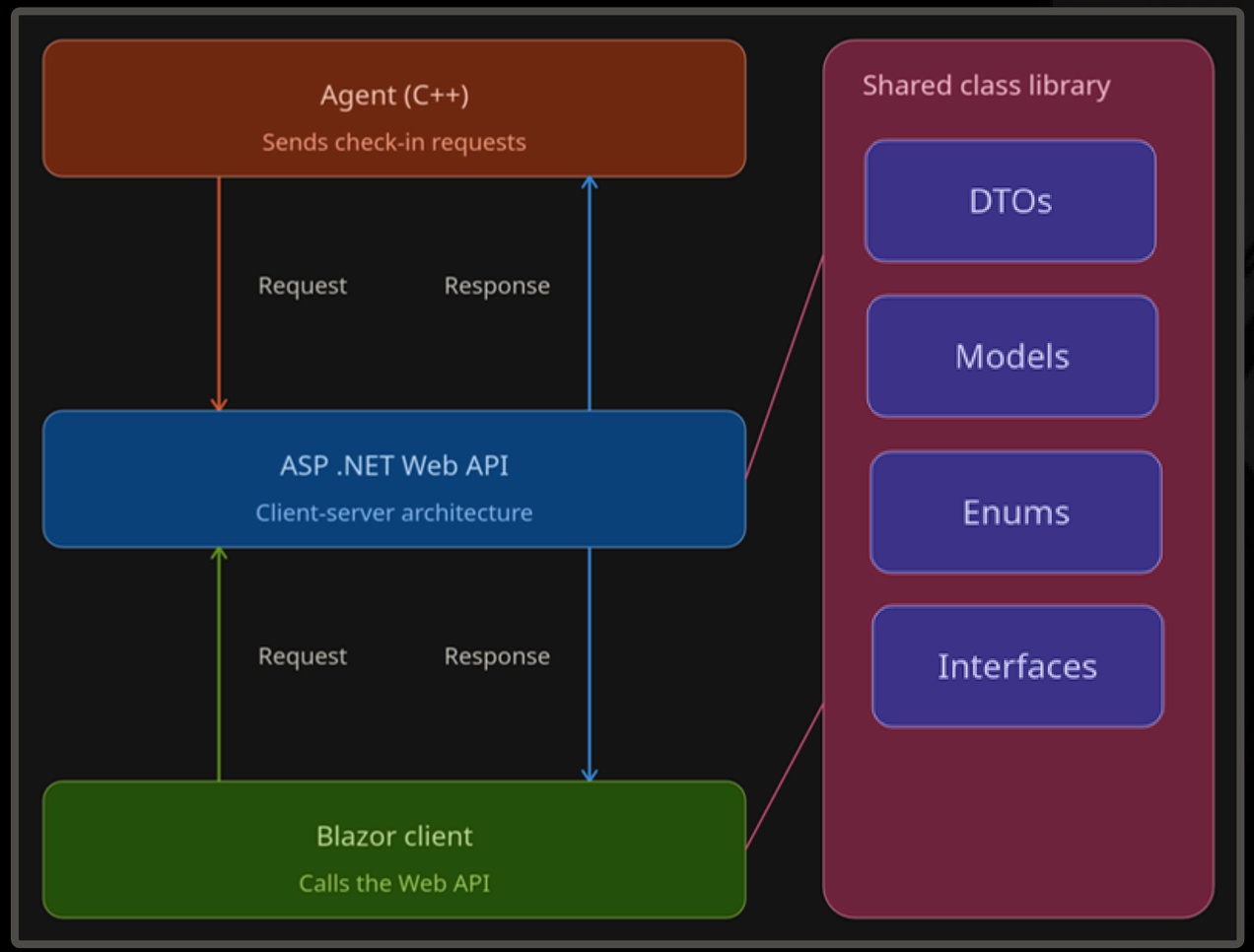
KEY FEATURES & FUNCTIONALITY

Agent

- **Command Execution:** Running shell commands natively or spawning process routines (cmd.exe, and powershell).
- **File Operations:** Locates, reads, and transfers the specified target file from the compromised host directly back to the Teamserver storage.
- **In-Memory Execution:** Running code (Such as .NET assemblies, object files and using LOLBINS) directly in memory without touching the disk to avoid file-based AV detection.
- **Extensibility (OF/Plugins):** Support for Object Files (OFs) or small compiled C plugins that execute within the agent's process context to keep the binary small and lightweight.
- **Host enumeration:** Extracting local host information, such as the Windows version, hardware, network interface, and process related data.



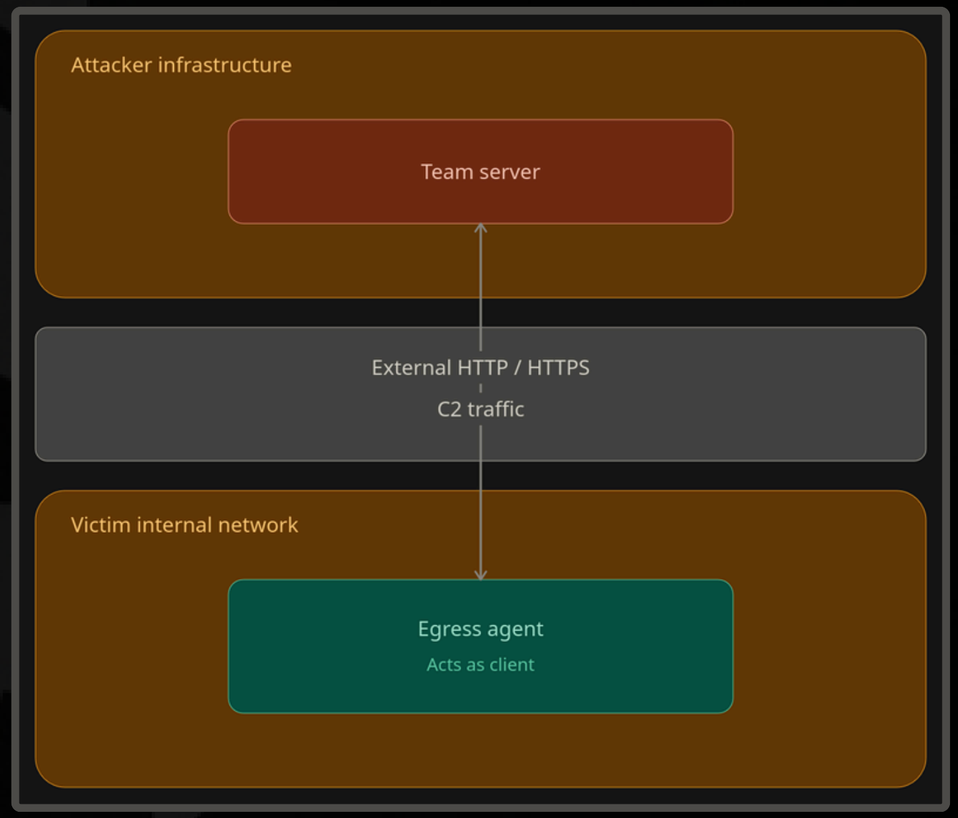
PROJECT STRUCTURE



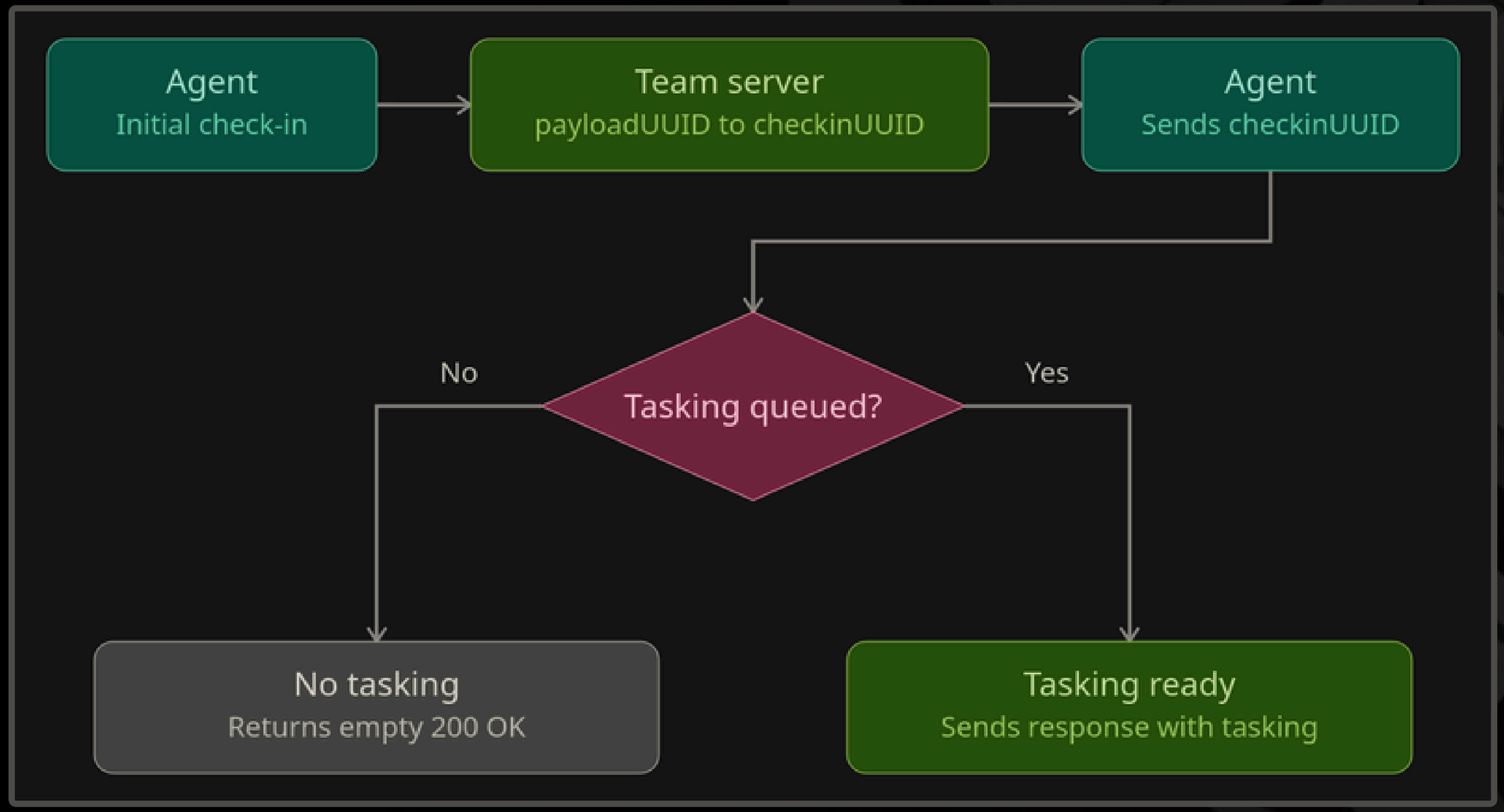
SYSTEM ARCHITECTURE & DESIGN DECISIONS

Client-Server Architecture

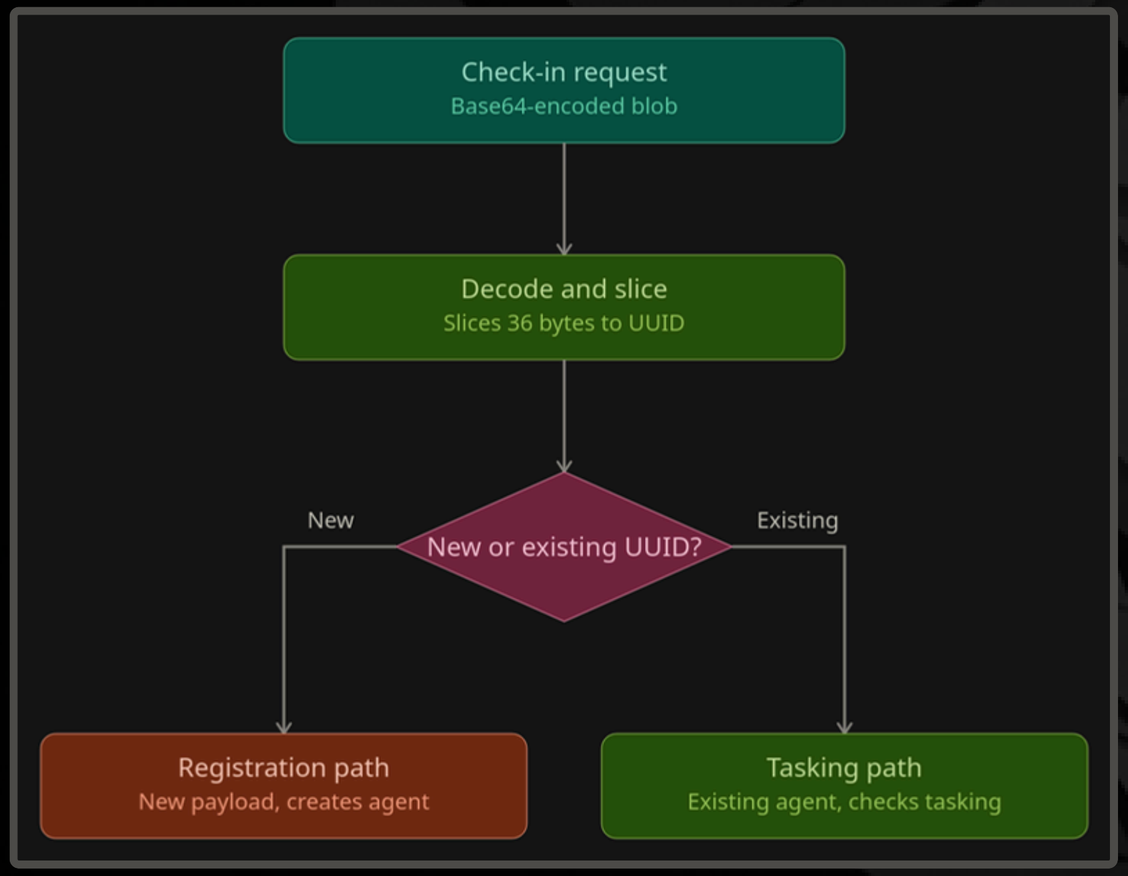
- **Decoupled Operation:** The Blazor Client never talks to the C2 Agent directly. The Teamserver sits in the middle as the absolute single source of truth.
- **Asynchronous Flow:** Operators stage commands in the backend queue via the UI, which sit idle until the remote agent independently calls home to fetch them.
- **The Check-In Loop:** The Agent initiates all network traffic (pull model) to check for tasks, execute them locally, and send raw results back to the server listener.
- **Metadata-Driven Handshake:** The agent initiates contact using a unique payloadUUID to transmit critical host metadata (OS, IP, user privileges) to the Teamserver.
- **Dynamic Session Transition:** Upon validation, the Teamserver registers the implant and returns a unique callback session ID used exclusively for all subsequent tasking checks.



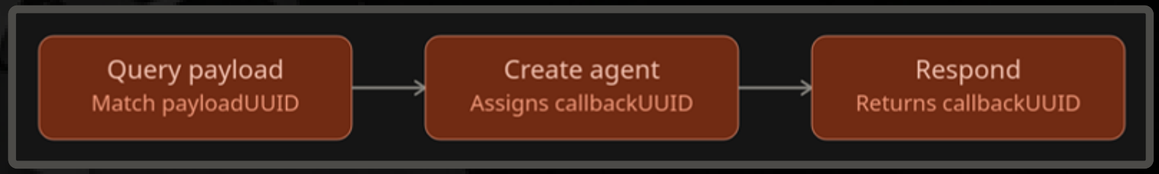
CHECKIN AGENT WORKFLOW



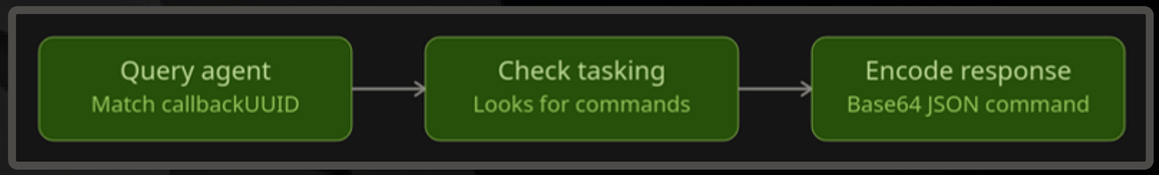
TEAM SERVER CHECKIN PROCESSING



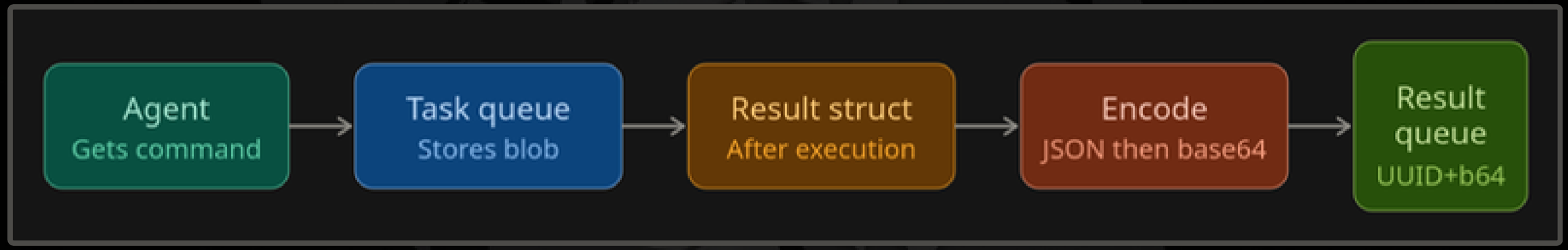
If it's a payloadUUID, the server registers the agent:



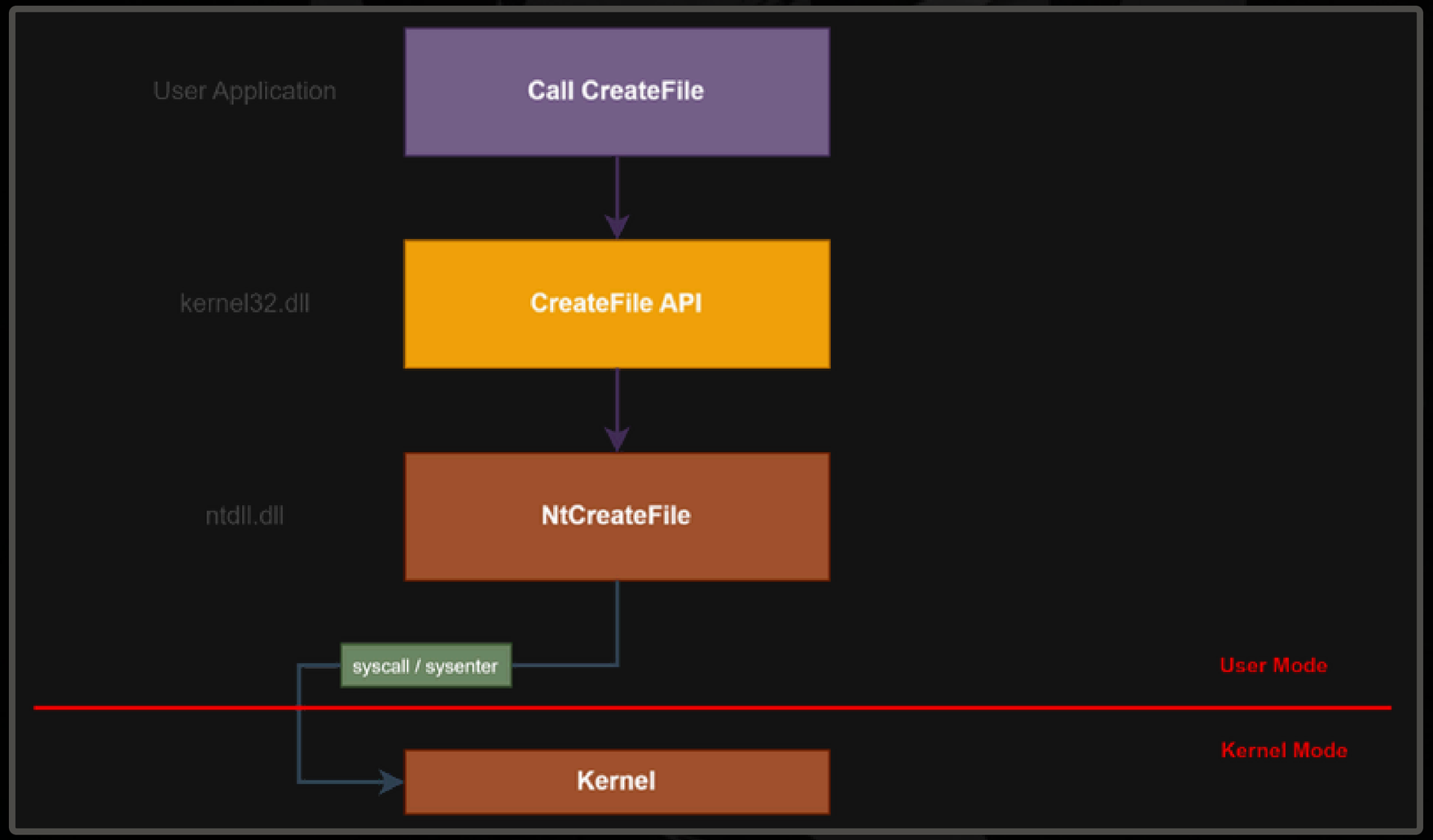
If it's an existing callbackUUID, the team server looks up tasking instead:



AGENT TASKING WORKFLOW



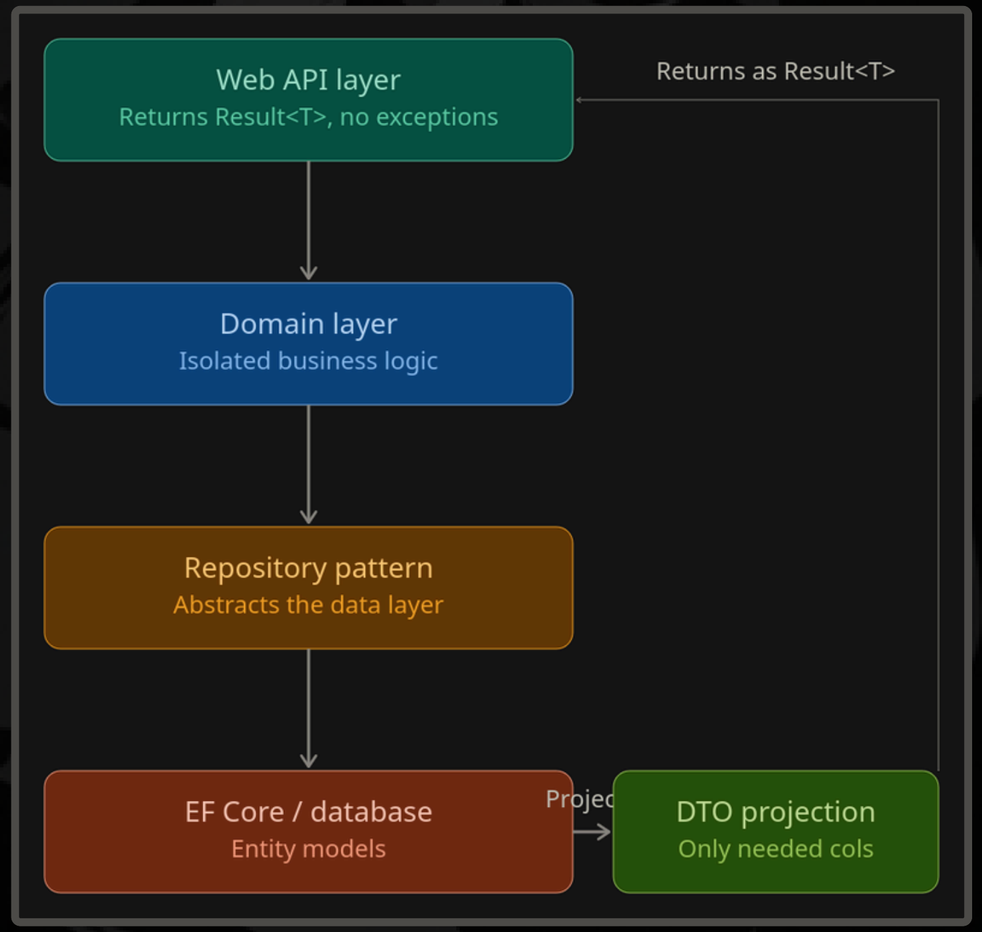
AGENT COMMAND EXECUTION



SYSTEM ARCHITECTURE & DESIGN DECISIONS

Application Programming Interface (API) Design

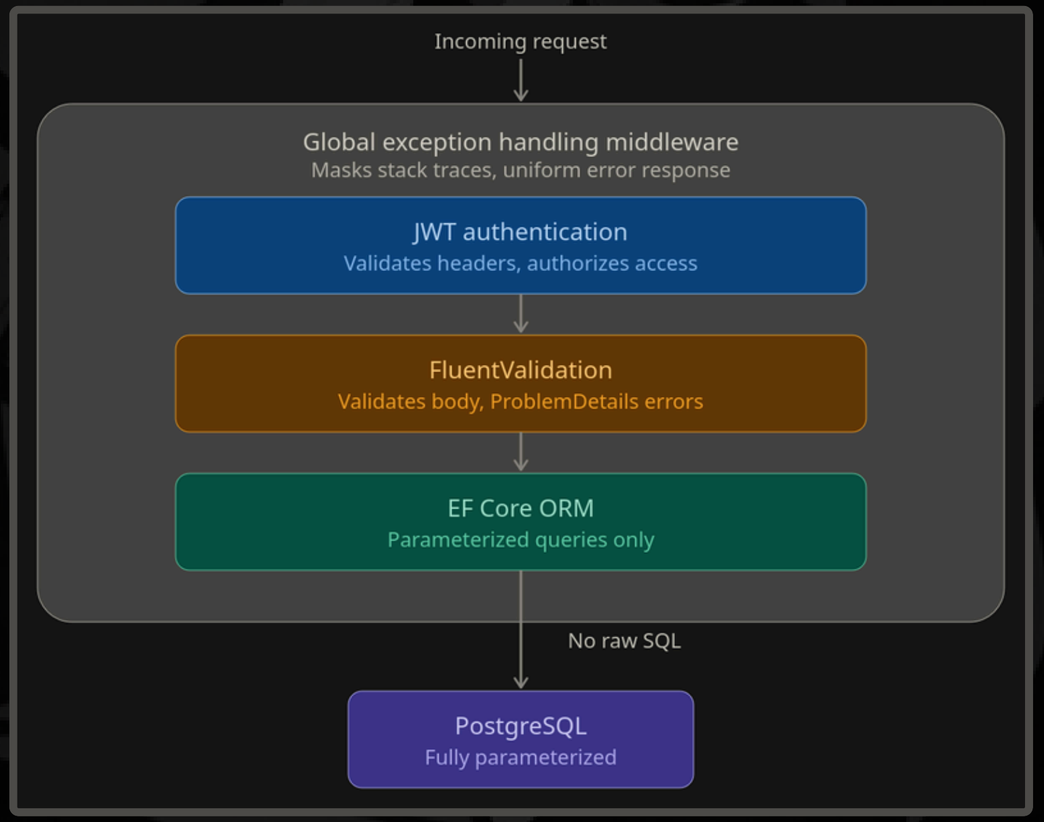
- **The Result Pattern:** The ASP.NET Web API completely avoids throwing costly runtime exceptions for business logic by returning a unified Result<T> or Result (non-generic) object that explicitly maps operation success or predictable error states.
- **Domain & Repository Isolation:** The back-end enforces a strict separation of domain boundaries, using the Repository Pattern to abstract the data layer and decouple core business logic from database engine dependencies.
- **Efficient EF Core Projections:** To maximize performance, the system maps entity models directly into lightweight Data Transfer Objects (DTOs) via EF Core projections, retrieving only the precise database columns needed for each API response.



SYSTEM ARCHITECTURE & DESIGN DECISIONS

API Security Architecture

- **SQL Injection Prevention:** Database access via PostgreSQL relies strictly on Entity Framework Core ORM and fully parameterized SQL queries, completely prohibiting concatenated raw SQL execution across all service handlers.
- **JWT Authentication:** Secure token-based authentication guards all API endpoints, validating incoming headers to authorize operator access .
- **Fluent Validation Pipelines:** All incoming POST body models are automatically intercepted and validated using FluentValidation, short-circuiting malformed requests early and returning standardised ProblemDetails error formats.
- **Global Exception Handling:** A centralised middleware handler catches unhandled runtime errors, masking internal system stack traces from malicious discovery and returning uniform bad request responses to callers.



SYSTEM ARCHITECTURE & DESIGN DECISIONS

Future CQRS Integration (Command Query Responsibility Segregation)

- **Architectural Segregation:** The system will transition to the CQRS pattern, completely separating data modification writes (Commands) from read-only data fetches (Queries).
- **Decoupled DTO Population:** This structural split isolates the repository layer from presentation requirements, ensuring that read queries project database entities directly into optimized DTO models without cluttering core data access code.
- **Optimized Performance & Scaling:** By decoupling the read and write pipelines, the API can scale data retrieval pathways independently, laying the groundwork for high-throughput, near real-time agent tracking and logging operations.

SYSTEM ARCHITECTURE & DESIGN DECISIONS

Areas of concern

- **Real-Time State Consistency:** The back-end must maintain consistent task state when multiple operators interact with the same agent, command queues and execution results update across all connected client UIs in near real-time without collisions.

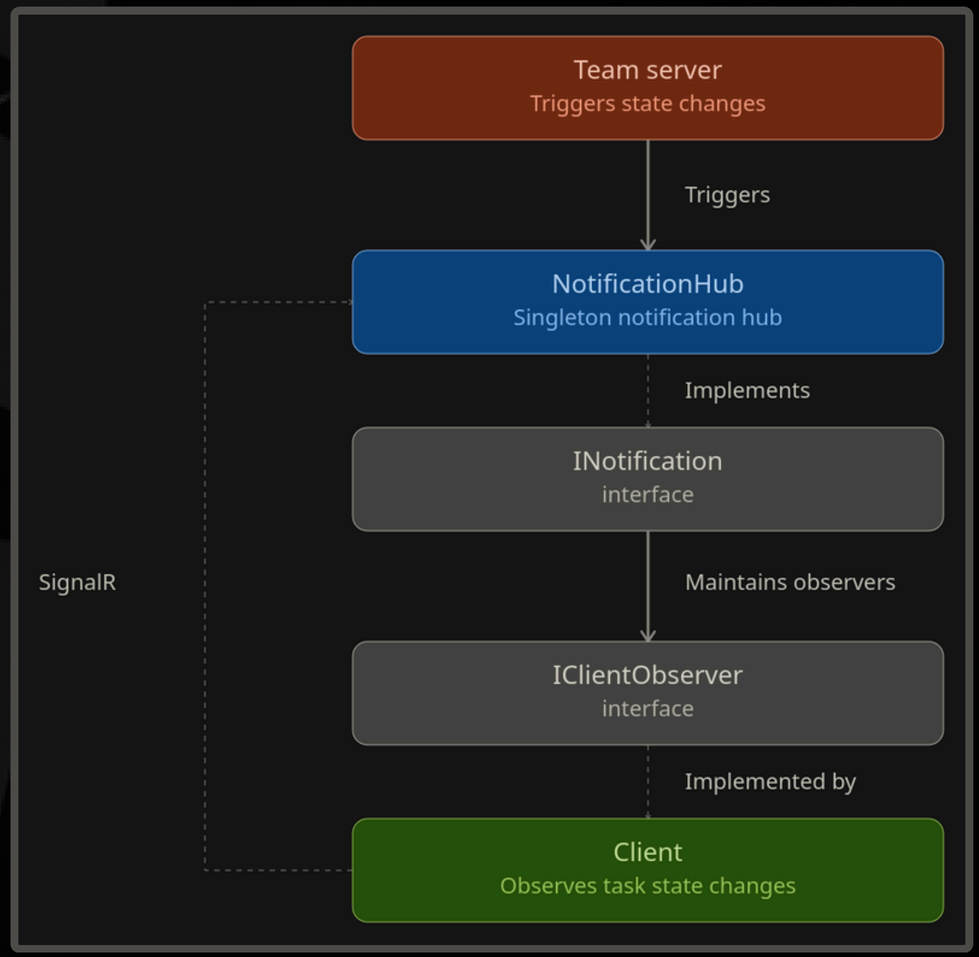
- **Heterogeneous Listener Abstraction:** The system must provide a unified interface for all listener objects while managing entirely different configuration schemas, network behaviors, and underlying implementation logic across distinct transport protocols (like HTTP, TCP, and SMB).

- **High Command variance:** The client UI must accept custom operator inputs and map them directly into Command Data Transfer Objects (DTOs) tailored to each specific command type, abandoning a generalised structure due to the extreme variance in command arguments.

ADDRESSING THESE CONCERNS

Real-Time State Consistency & The Observer Pattern

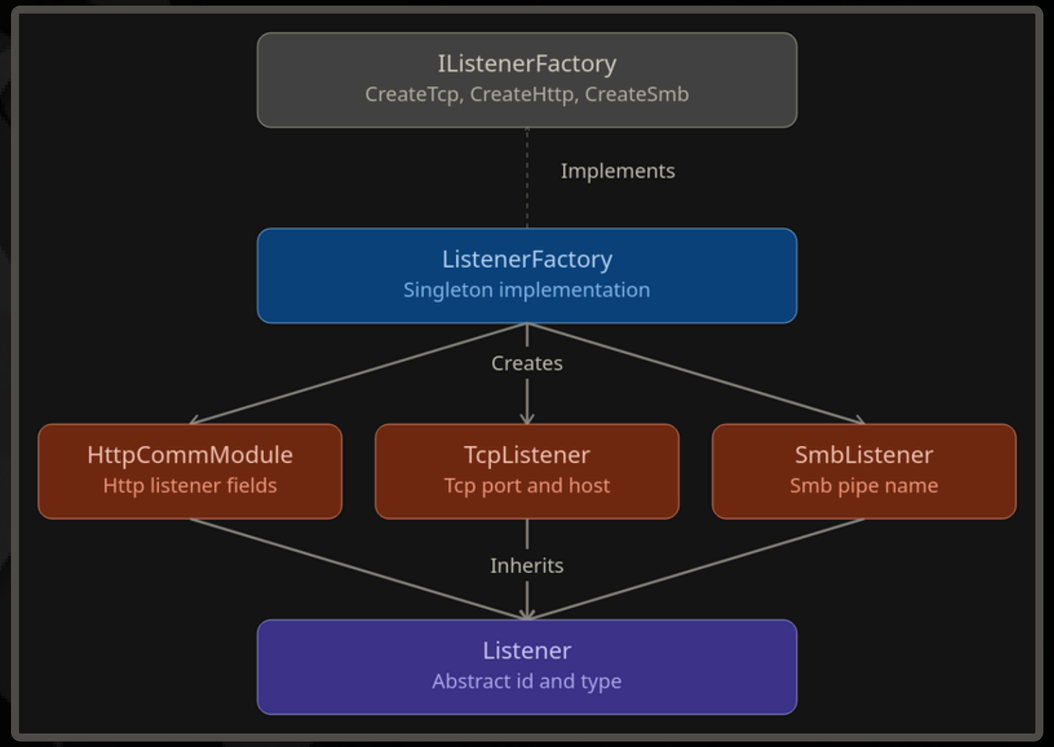
- **The Architectural Challenge:** The system must support multiple concurrent campaigns while maintaining near real-time task state consistency across an unpredictable number of connected clients, dealing with a strict one-to-many dependency between the Teamserver and its users.
- **The Observer Solution:** To decouple components and eliminate data desynchronization, the system implements the Observer Design Pattern, which automatically triggers runtime updates to all active, interested client dependencies the moment a task state changes.
- **SignalR Notification Hub:** A singleton NotificationHub manages these connections dynamically—attaching clients on login and detaching them on timeout—using a unified interface to push live updates downstream to all subscribed operators simultaneously.



ADDRESSING THESE CONCERNS

Listener Creation & The Abstract Factory Pattern

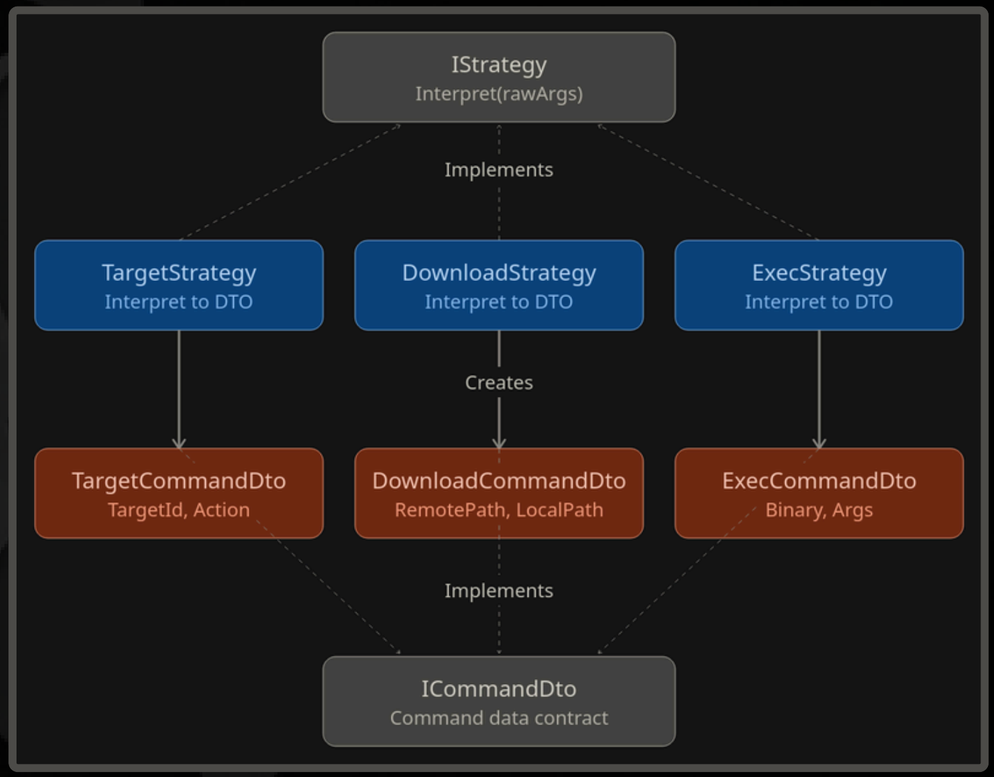
- **The Creational Challenge:** The Teamserver must build different types of listeners (HTTP, TCP, SMB) that share a common conceptual goal but require vastly different configuration schemas, network behaviors, and initialization logic.
- **The Conventional Factory Pitfall:** Standard factories rely on complex conditional type-checking, which leaks protocol dependencies to the caller and exposes internal implementation details.
- **The Abstract Factory Solution:** A unified IListenerFactory interface defines a family of creation methods (CreateHttp(), CreateTcp(), CreateSmb()), completely encapsulating the distinct creation logic into a singleton factory accessible via dependency injection.



ADDRESSING THESE CONCERNS

Command Processing & The Strategy Pattern

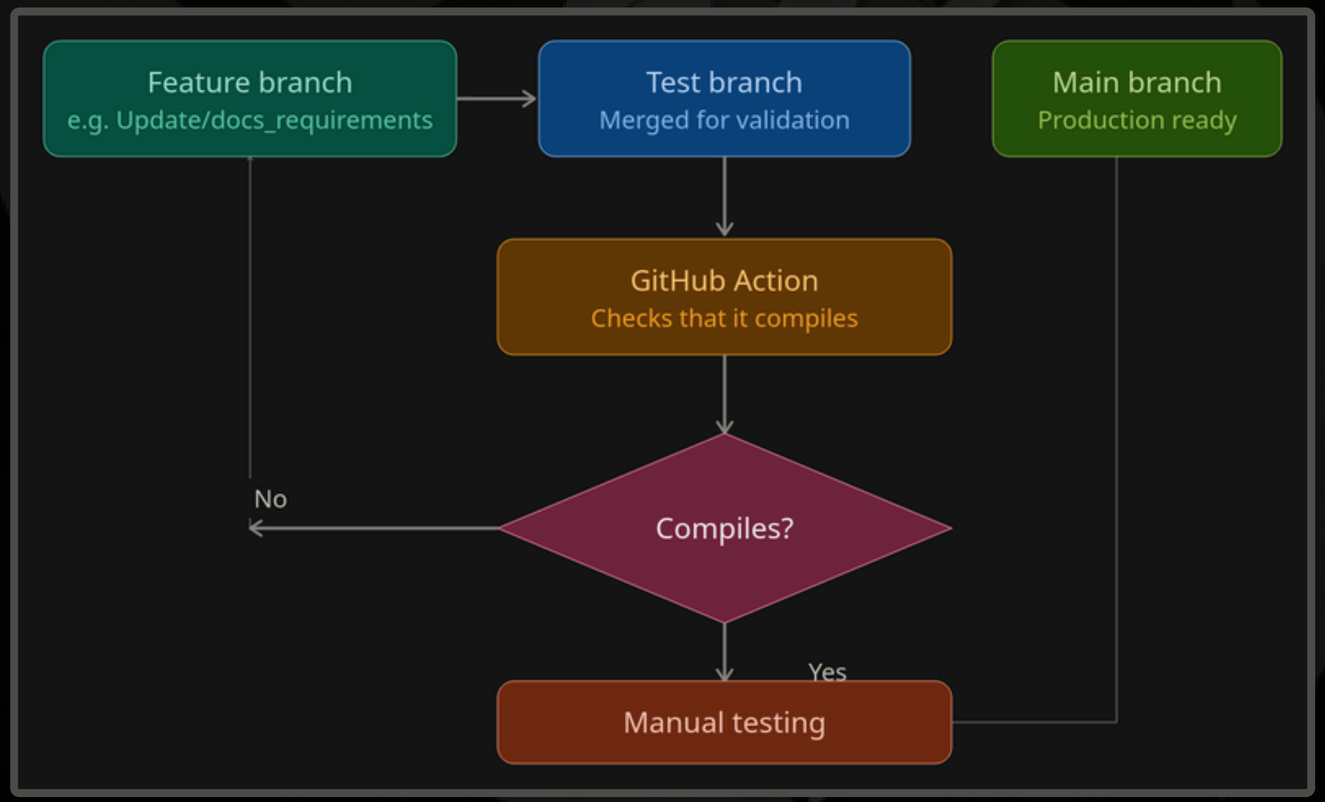
- **The Parsing Challenge:** The client UI must process highly varied operator commands and map them to specific Data Transfer Objects (DTOs), making a generalised input structure impossible.
- **The Architectural Risk:** Hard-coding different argument-parsing algorithms directly into presentation components (like the console terminal) introduces unnecessary complexity and leaks backend implementation details into the UI layer.
- **The Strategy Solution:** A CommandResolver identifies the command keyword and dynamically swaps interchangeable parsing algorithms via an IStrategy interface, cleanly isolating complex DTO mapping logic from the presentation view.



GITHUB VERSION CONTROL

Version Control and Branching Strategy

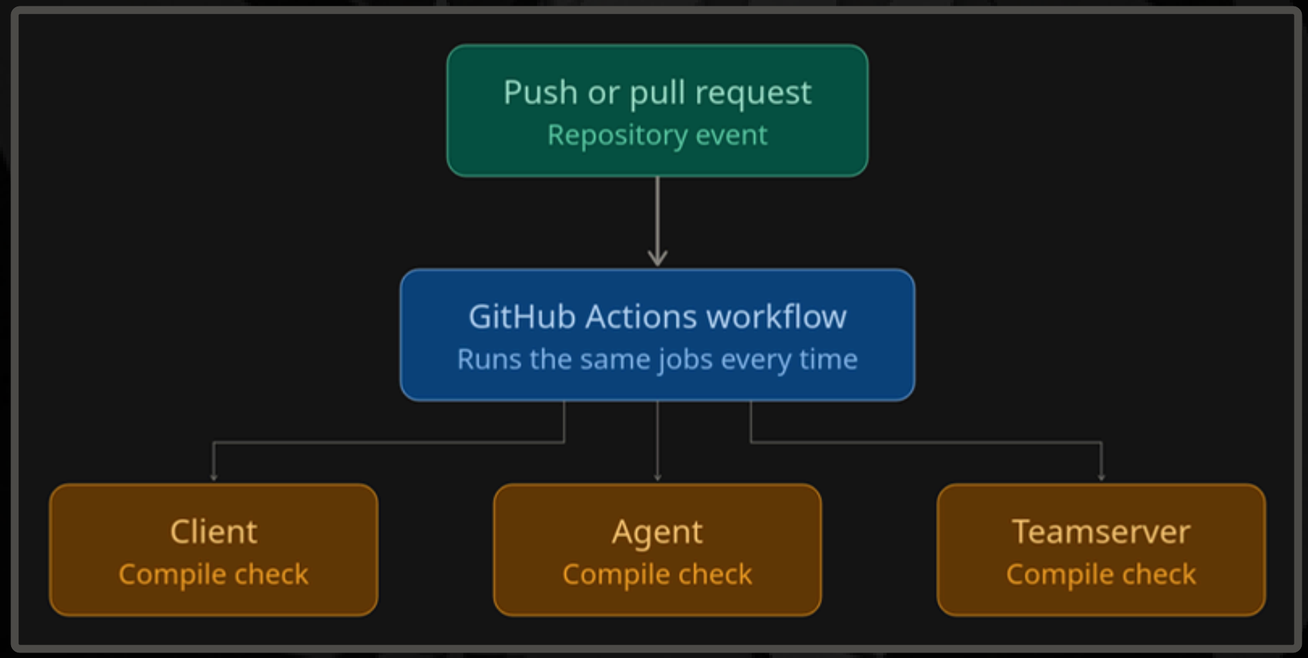
- **Isolated Feature Branching:** Developers isolate code base changes and updates into dedicated branches using a strict kanban-number-feature-name naming convention to keep modifications independent and reviewable.
- **The Deployment Pipeline:** Finished feature branches are first merged into a dedicated test branch, triggering a automated GitHub Action to verify successful code compilation before any manual testing begins.
- **Production Main Merge:** Changes are only merged into the main branch after passing both automated compilation checks and manual verification, ensuring a clean history and a stable production project.



GITHUB VERSION CONTROL

Automated testing and CI pipeline

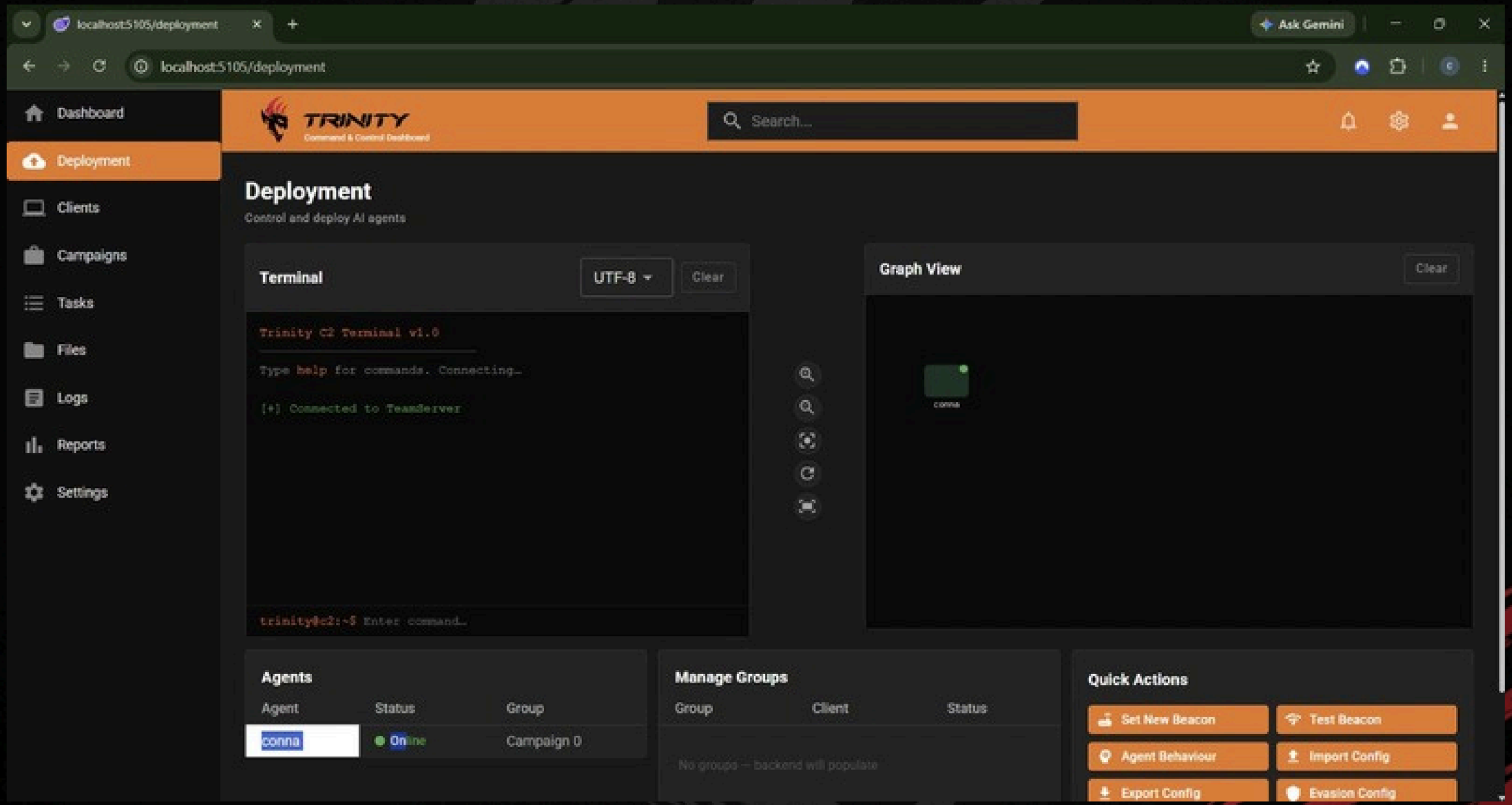
- **Continuous Integration Automation:** The system utilises GitHub Actions workflows to enforce consistent build and quality assurance procedures across the entire team, entirely eliminating environment-specific deployment discrepancies.
- **Event-Driven Ingestion:** Automated jobs are instantly triggered by repository events, specifically targeting every code push and pull request submitted by our team.
- **Cross-Component Compilation Check:** The pipeline mandates that all foundational ecosystem components, including the client, agent, and teamserver, compile successfully before changes can advance to production.



TRINITY C2 FUNCTIONAL PROTOTYPE

DEMONSTRATION

View here


localhost:5105/deployment
Ask Gemini
localhost:5105/deployment
Dashboard
Deployment
Clients
Campaigns
Tasks
Files
Logs
Reports
Settings
TRINITY
Command & Control Dashboard
Search...
Deployment
Control and deploy AI agents
Terminal
UTF-8
Clear
Trinity C2 Terminal v1.0
Type help for commands. Connecting…
[+] Connected to Teamserver
trinity@c2:~$ Enter command…
Graph View
Clear
conna
Agents
Agent
Status
Group
conna
Online
Campaign 0
Manage Groups
Group
Client
Status
No groups — backend will populate
Quick Actions
Set New Beacon
Test Beacon
Agent Behaviour
Import Config
Export Config
Evasion Config

CODE WALKTHROUGH

AGENT CAPABILITIES (INTIAL AND FULL CHECKIN), LISTENER AND CHECKIN API SERVICES

View here

James Marshall (ST10434366)

CODE WALKTHROUGH

AGENT CAPABILITIES (KEYLOGGER, POWERSHELL), ADMIN, FILE, PAYLOAD, AND OPERATOR API SERVICES

View here

Lyle William Richards (ST10322521)

CODE WALKTHROUGH

BLAZOR CLIENT PROJECT (USER INTERFACES)

View here

Connaire Myall (ST10297849)

CODE WALKTHROUGH

BLAZOR CLIENT PROJECT (USER INTERFACES)

View here

Simon Slingerland (ST10438897)

James Marshall (ST10434366), Lyle William Richards (ST10322521), Sky Martin (ST10286905), Connaire Myall (ST10297849), and Simon Slingerland (ST10438897)

05 October 2026

0xFFFF